Table of Contents

Week 3: Project Proposal & Planning 2
Project proposal 2
Title 2
Problem statement 2
Aim and objectives 2
Scope and key features 3
Gantt plan (six weeks) 3
Risk register 4
Week 4: Requirements & Use Case Modelling 4
Selected user stories 4
Detailed use case descriptions 5
Use Case 1: Submit a Service Request (US02) 5
Use Case 2: View My Service Requests (US03) 6
Use Case 3: Update Request Status (US05) 7
UML use case diagram 8

INDIVIDUAL ASSIGNMENT [100MARKS]

Question

FIND A PROBLEM THAT EXISTS IN THE WOMENS UNIVERSITY ECOSYSTEM.THE PROBLEM MUST BE SOLVED COMPUTATIONALLY.APPLY THE FOLLOWING SECTIONS AND SOLVE THE PROBLEM.

Week 1: Setting up Development Environments & Git

Lab 1: Git Basics & Environment Setup

- Install Git, VS Code (or PyCharm), and create a GitHub account.
- Configure Git (user.name, user.email).
- Create a local repo, add a README.md, commit, and push to GitHub.

- Practice: Clone a repo, make a change, commit, and push.

Week 2: Team Project Kick-off & Product Backlog

Lab 2: Creating a Product Backlog in GitHub Projects

- Create a GitHub repo for your team project.
- Use GitHub Issues or Trello to create a **product backlog** (10–15 user stories).
- Label stories as Must-have, Should-have, Could-have.
- Prioritize using MoSCoW method.

Week 3: Project Proposal & Planning

Lab 3: Writing a Mini Project Proposal

Project proposal

Title

WUA Student Support and Service Request System (SSRS)

Problem statement

Routine university service requests can involve several parties: a student raises an issue, a service unit receives it, a staff member investigates it, and the student needs to know whether it has been resolved. If the process relies on disconnected messages or informal follow-ups, requests may be difficult to trace and staff may lack a shared view of their status. This is a problem hypothesis, not a confirmed claim about WUA. Short interviews with students and service staff will be used to establish whether the problem exists locally and which request types are most affected.

Aim and objectives

Aim:

1. Design and prototype a secure, traceable digital system for routine student service requests.
2. Provide a structured form for submitting a request and generating a unique reference number.
3. Allow students to view the status and history of their own requests.
4. Allow authorized staff to assign, update, and close requests.

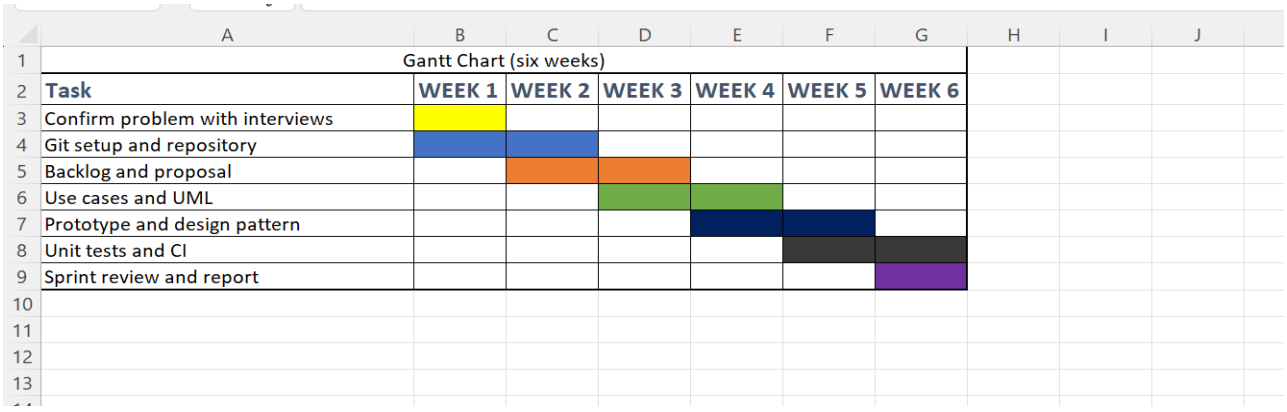
5. Protect student information through role-based access and minimal data collection.
6. Use automated tests and a continuous integration workflow to detect regressions.

Scope and key features

In scope: authentication prototype, request submission, request listing, status updates, category selection, timestamps, and basic workload counts. Out of scope for the first release: emergency response, payments, medical records, confidential counselling case management, and production integration with university systems.

Gantt plan (six weeks)

Task	W1	W2	W3	W4	W5	W6
Confirm problem with interviews	●					
Git setup and repository	●	●				
Backlog and proposal		●	●			
Use cases and UML			●	●		
Prototype and design pattern				●	●	
Unit tests and CI					●	●
Sprint review and report						●



Risk register

Risk	Likelihood / impact	Mitigation
The problem hypothesis is not confirmed.	Medium / High	Conduct short interviews first; adjust the project based on evidence.
Unauthorized users view another student's requests.	Medium / High	Enforce server-side ownership checks and role permissions; test access-control cases.
Scope grows beyond available time.	High / Medium	Deliver Must-have features first; defer Could-have items.
Internet or device access is unreliable.	Medium / Medium	Keep the prototype lightweight and test on mobile-sized screens; document local setup.
Loss of code or accidental secret exposure.	Medium / High	Push regularly to GitHub; use .gitignore; never commit credentials or real personal data.

Week 4: Requirements & Use Case Modelling

Lab 4: Writing Use Cases & Drawing UML Use Case Diagram

Selected user stories

ID and title	Primary actor	Purpose
US02 — Submit a Service Request	Student	Submit a routine service issue in a structured form.
US03 — View My Service Requests	Student	View and track only the student's own submitted requests.
US05 — Update Request Status	Authorized service staff	Record request progress so that status information remains current and traceable.

Detailed use case descriptions

Use Case 1: Submit a Service Request (US02)

Field	Description
Use case ID	UC01
Use case name	Submit a Service Request
Related user story	US02
Primary actor	Student
Goal	Submit a routine service issue and receive a reference number and confirmation.
Preconditions	1. Student has an active account and is signed in. 2. At least one request category is available. 3. The system can accept and store requests.
Trigger	Student selects “Submit a Service Request”.
Postconditions — success	A request is saved with a unique reference number, owner, category, description, timestamp, and initial status “Submitted”. A confirmation is displayed.
Postconditions — failure	Invalid data is not saved as a completed request. The system does not show a success message or reference number for an unsaved request.

Main success flow

1. Student opens the new service request page.
2. System displays the form and available categories.
3. Student selects a category and enters a description.
4. Student submits the form.
5. System validates required fields and checks description length/content.
6. System creates the request and generates a unique reference number.
7. System saves the request with status “Submitted” and records the date/time.
8. System displays confirmation and the reference number.

Alternative and exception flows

ID	Condition	System response
A1	A required field is missing.	Highlight the missing field, explain what is required, and do not create the request. The student can correct the form.

A2	Description is blank, whitespace-only, or too long.	Display a validation message and ask the student to revise the description.
A3	Student session has expired.	Require sign-in again; do not submit under an unidentified account.
A4	Saving fails because of a system/database error.	Display a safe error message, do not show a reference number for an unsaved request, and allow a retry.
A5	No categories are available.	Explain that submission is temporarily unavailable and provide an appropriate next step.

Use Case 2: View My Service Requests (US03)

Field	Description
Use case ID	UC02
Use case name	View My Service Requests
Related user story	US03
Primary actor	Student
Goal	View submitted requests and check their current statuses.
Preconditions	1. Student is signed in. 2. System can retrieve request data.
Trigger	Student selects “My Service Requests”.
Postconditions — success	The system displays the student’s own requests and summary details. No request data is changed.
Postconditions — exception	If retrieval fails, the system reports the problem without exposing another student’s information.

Main success flow

1. Student opens “My Service Requests”.
2. System identifies the student from the authenticated session.
3. System retrieves requests owned by that student only.
4. System displays each request’s reference number, category, submission date, and current status.
5. Student selects a request to view its details.
6. System displays permitted details and student-visible update history.
7. Student reviews the status and may return to the list.

Alternative and exception flows

ID	Condition	System response

A1	Student has not submitted any requests.	Display an empty-state message and offer the option to submit a request.
A2	Student attempts to open a request owned by another student.	Deny access without revealing request details. Enforce the ownership check on the server, not only in the interface.
A3	Session has expired.	Require sign-in before displaying request information.
A4	Request list cannot be loaded.	Display a clear temporary error and a retry option.
A5	Reference is invalid or unavailable to the student.	Display a general “Request not found or unavailable” message without disclosing private information.

Use Case 3: Update Request Status (US05)

Field	Description
Use case ID	UC03
Use case name	Update Request Status
Related user story	US05
Primary actor	Authorized service staff member
Goal	Record the latest progress of a service request.
Preconditions	1. Staff member is signed in. 2. Staff member has permission to manage the request. 3. The request exists.
Trigger	Staff member opens a request and selects a new status.
Postconditions — success	New status and timestamp are saved. History records the change and staff identity. The student can see the student-visible status.
Postconditions — failure	An unauthorized or invalid update is not saved; the previous valid status remains unchanged.

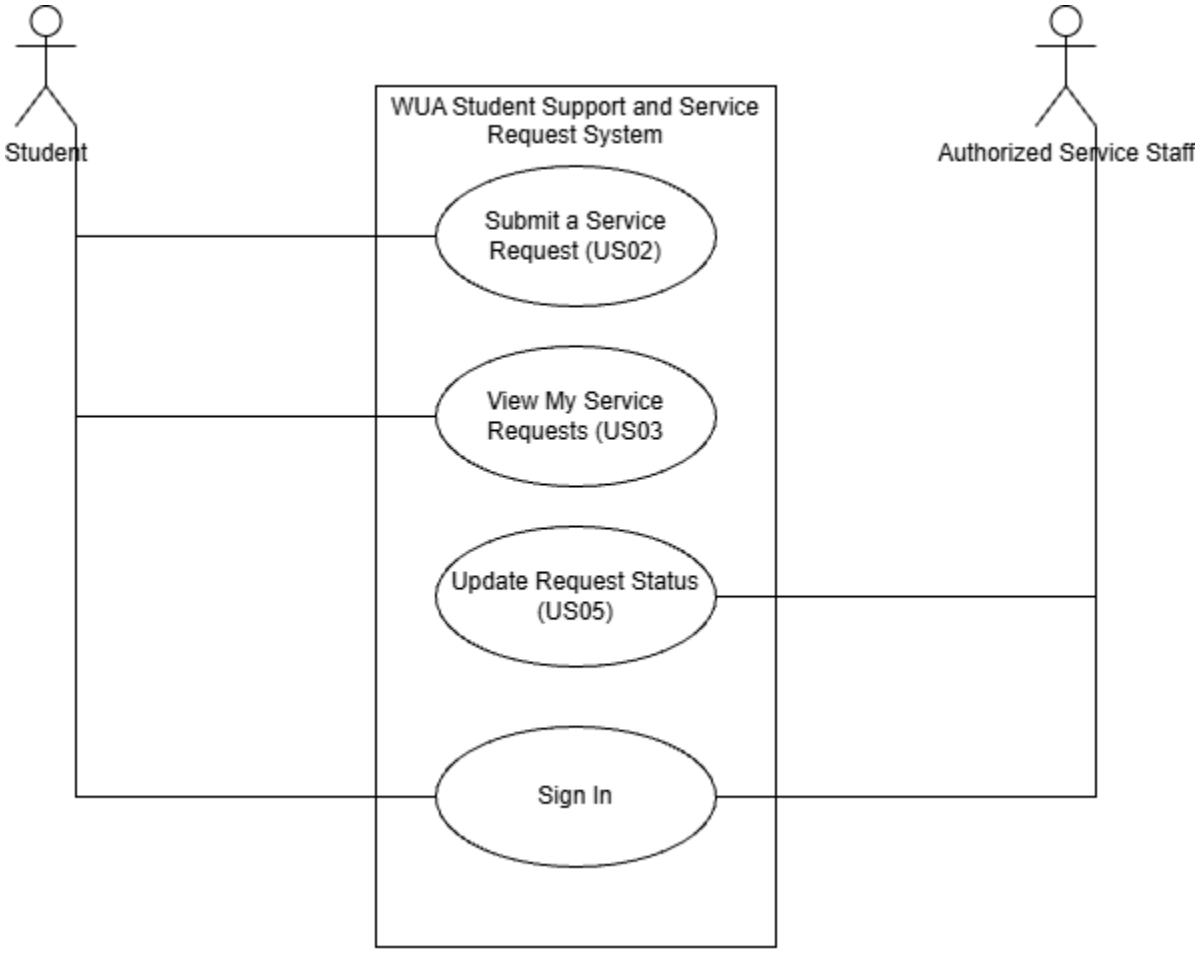
Main success flow

1. Staff member opens the staff request dashboard.
2. System displays requests the staff member is authorized to access.
3. Staff member selects a request and reviews its details.
4. Staff member chooses a permitted status, such as “In Progress”, “Awaiting Information”, “Resolved”, or “Closed”.
5. System verifies staff permissions and validates the status transition.
6. System saves the status and records date/time and staff identity in the request history.
7. System confirms the update succeeded.
8. Updated status becomes visible to the student in “My Service Requests”.

Alternative and exception flows

ID	Condition	System response
A1	User lacks staff permission.	Reject the update and leave the request unchanged.
A2	Selected status is not permitted from the current status.	Explain that the transition is invalid and present permitted choices.
A3	Request no longer exists or cannot be accessed.	Display an unavailable-request message and return staff member to the list.
A4	Saving the update fails.	Report that the update was not saved; do not show success or create a misleading history entry.
A5	Another staff member updates the request at the same time.	Reload the latest status and ask the staff member to review and retry rather than silently overwriting the newer update.

UML use case diagram



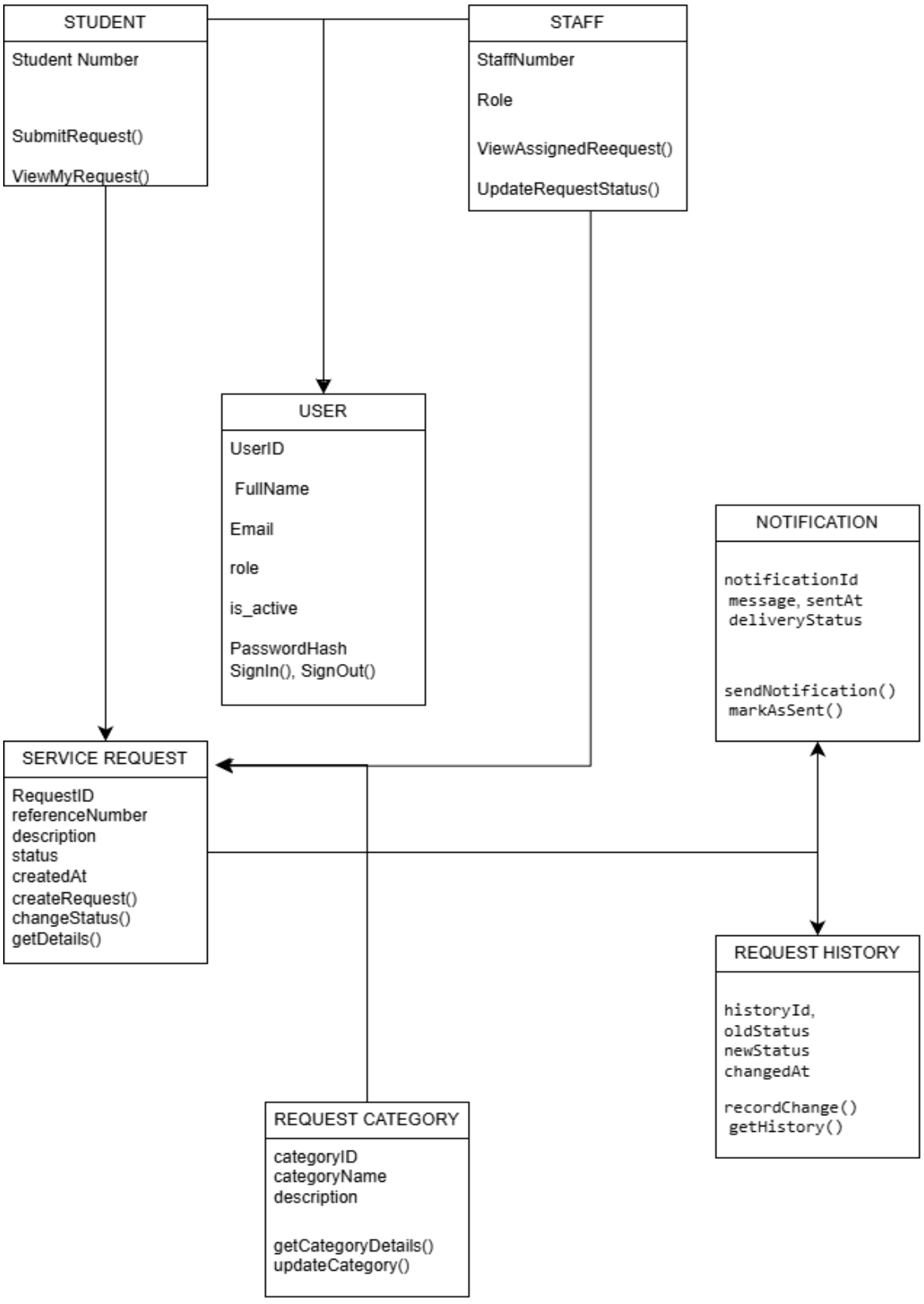
Week 5: Class & Sequence Diagrams

Lab 5: From Use Cases to Class Diagrams

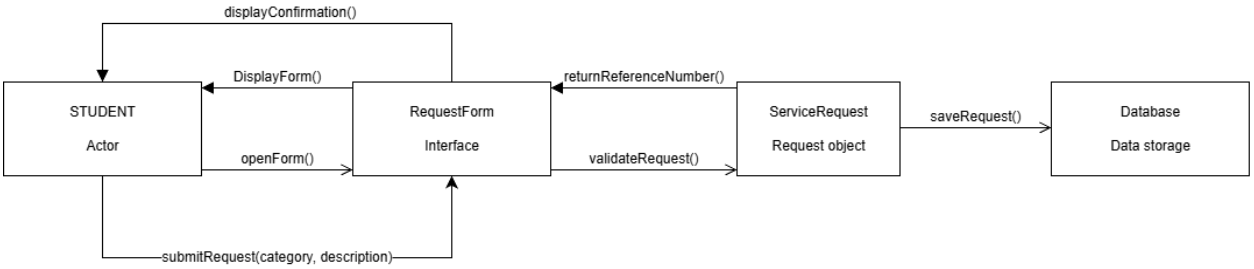
Candidate classes

Class	Example attributes	Main methods / responsibilities
User	user_id, name, email, role, is_active	authenticate(), has_permission()
Student	student_number	submit_request(), list_own_requests()
Staff	staff_id, service_unit	view_assigned(), update_status()
ServiceRequest	request_id, reference, category, description, status, created_at, owner_id, assignee_id	validate(), change_status(), generate_reference()
RequestCategory	category_id, name, is_active	activate(), deactivate()
RequestUpdate	update_id, request_id, actor_id, old_status, new_status, note, created_at, visible_to_student	record_update()
Feedback	feedback_id, request_id, rating, comment, created_at	validate_feedback()

UML class diagram



UML sequence diagram



Week 6: Architectural Design

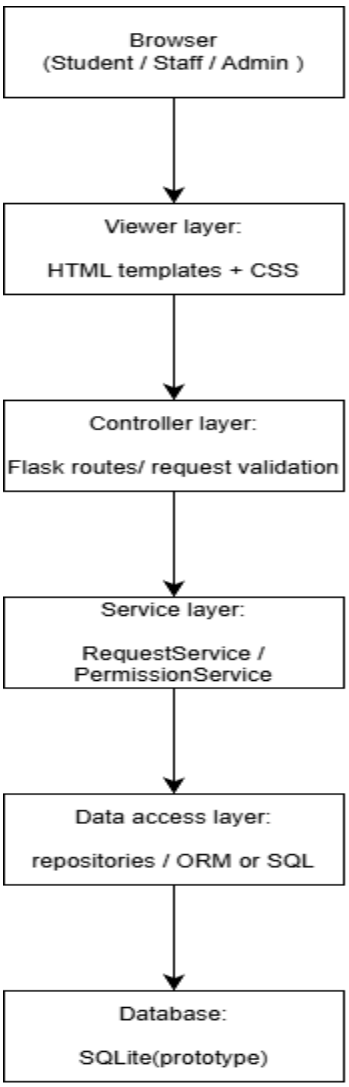
Lab 6: Choosing & Documenting Architecture

Chosen architecture: MVC architecture

Model–View–Controller (MVC), implemented as a small Flask application. MVC separates data and business rules (Model), user interface (View), and request handling (Controller). For a student-sized prototype, a modular monolith is simpler to test and deploy than microservices. The system can be split into services later if real usage and operational needs justify it.

Requirement	Design response
Security and privacy	Role checks on the server; ownership checks for student records; password hashing if real authentication is implemented; no sensitive data in logs.
Maintainability	Separate routes/controllers, service logic, data models/repository, templates, and tests.
Usability	Responsive pages, clear validation, status labels, and reference numbers.
Reliability	Database transactions, error handling, automated tests, version control.
Scalability	Start with a modular monolith and SQLite for the local prototype; use a managed production database only after requirements and deployment are approved.

Component/package diagram



Week 7: Implementing a Design Pattern

Lab 7: Refactoring with a Design Pattern

Week 7: Refactoring and Design Patterns

Messy Code

```
system_name = "WUA SSRS" def

submit_request(student, description):

   request = {

      "student": student,

      "description": description,

      "status": "Submitted"

   }

   print("Request saved")


def update_status(request, new_status):

   request["status"] = new_status

print("Status changed")

print("Send notification to student")
```

What is wrong with it?

- Configuration is just a global variable.
- Request creation is mixed with other application logic and has no central validation.
- Status updating is directly tied to notification behaviour.
- The code becomes harder to extend as new features are added.

Python implementation

```
from dataclasses import dataclass

from datetime import datetime

from uuid import uuid4
```

```
# 1. SINGLETON PATTERN class
SystemConfig:     _instance =
None     def __new__(cls):
if cls._instance is None:
            cls._instance = super().__new__(cls)
cls._instance.system_name = "WUA SSRS"         return
cls._instance
# 2. FACTORY PATTERN @dataclass class
ServiceRequest:     reference: str
student_id: str     category: str
description: str     status: str     created_at:
str class ServiceRequestFactory:
@staticmethod     def create(student_id,
category, description):         if not
student_id.strip():
            raise ValueError("Student ID is required.")
if not category.strip():
            raise ValueError("Category is required.")
if not description.strip():             raise
ValueError("Description is required.")         return
ServiceRequest(             reference=f"REQ-
{uuid4().hex[:8].upper()}",
student_id=student_id,             category=category,
description=description,
status="Submitted",
created_at=datetime.now().isoformat(
timespec="seconds"
            ),
        ) # 3. OBSERVER PATTERN class
RequestStatusNotifier:     def __init__(self):
self._observers = []     def subscribe(self,
observer):         self._observers.append(observer)
def notify(self, request, old_status, new_status):
for observer in self._observers:
```

```
            observer(request, old_status, new_status)
def print_status_update(request, old_status,
new_status):
    print(         f"Request
{request.reference}: "
f"{old_status} -> {new_status}"
    ) # Demonstration if __name__ == "__main__":
config1 = SystemConfig()     config2 = SystemConfig()
print("Same config instance:", config1 is config2)
print("System:", config1.system_name)     request =
ServiceRequestFactory.create(
student_id="STU001",         category="Maintenance",
description="A classroom light is not working."
    )     print("Created request:", request)
notifier = RequestStatusNotifier()
notifier.subscribe(print_status_update)     old_status
= request.status     request.status = "In Progress"
notifier.notify(request, old_status, request.status)
```

Code component	Pattern	Purpose
SystemConfig	Singleton	Reuses one configuration instance.
ServiceRequestFactory	Factory	Creates service request objects in one place.
RequestStatusNotifier	Observer	Notifies listeners when a request status changes.

Refactoring and Design Patterns Reflection

During Week 7, I applied the Singleton, Factory and Observer design patterns to the proposed WUA Student Support and Service Request System (SSRS).

The Singleton pattern was used in the SystemConfig class to provide one shared configuration instance. This avoids creating multiple independent configuration objects. However, Singleton can introduce global-state and testing difficulties if used excessively.

The Factory pattern was implemented through ServiceRequestFactory. It centralises service request creation and validates essential fields before creating an object. This reduces duplicated creation logic and makes the process easier to maintain.

The Observer pattern was demonstrated using RequestStatusNotifier. When a service request's status changes, registered observers receive the old and new status values. This separates the status-change process from the actions triggered by that change, making it easier to add further listeners, such as an email notification service.

Overall, these patterns demonstrate how refactoring can improve code organisation, maintainability and extensibility. Nevertheless, design patterns should only be used where they solve a genuine design problem. The current implementation is a prototype and requires further testing, persistent data storage and appropriate security controls before it could be used in a real university environment.

Output



Week 8: Unit Testing with pytest/JUnit

Lab 8: Writing Unit Tests

request_validation.py:

```
def validate_request_description(description):
    """Return True for a non-empty description of at most 500 characters."""
    if not isinstance(description, str):
        return False
    cleaned = description.strip()
    return 1 <= len(cleaned) <= 500
```

test_request_validation.py

```
from request_validation import validate_request_description

def test_valid_description():
    assert validate_request_description("Wi-Fi is not working in the lab.") is True

def test_empty_description():
    assert validate_request_description("") is False

def test_spaces_only():
    assert validate_request_description("   ") is False

def test_maximum_length_is_accepted():
    assert validate_request_description("a" * 500) is True

def test_too_long_is_rejected():
    assert validate_request_description("a" * 501) is False

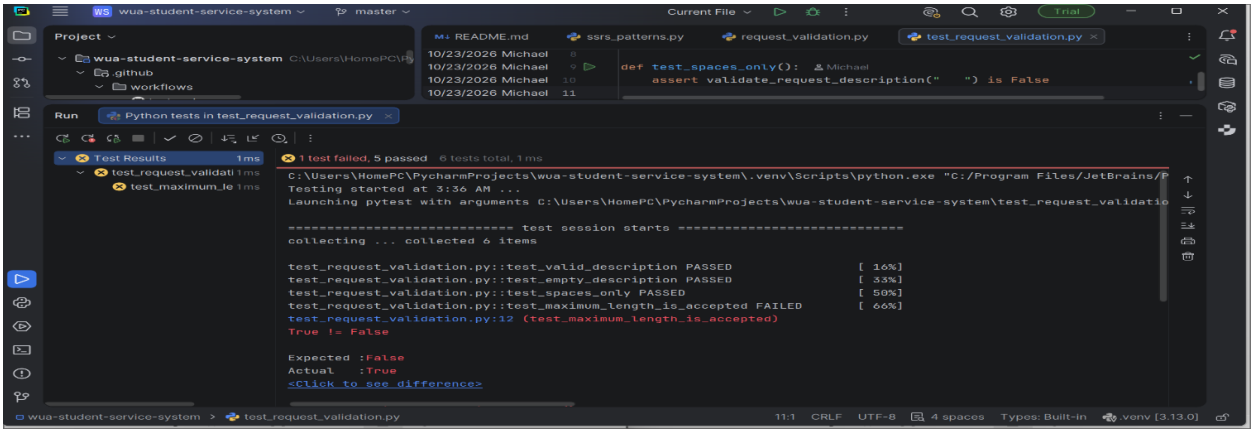
def test_non_string_is_rejected():
    assert validate_request_description(None) is False
```

Run the tests

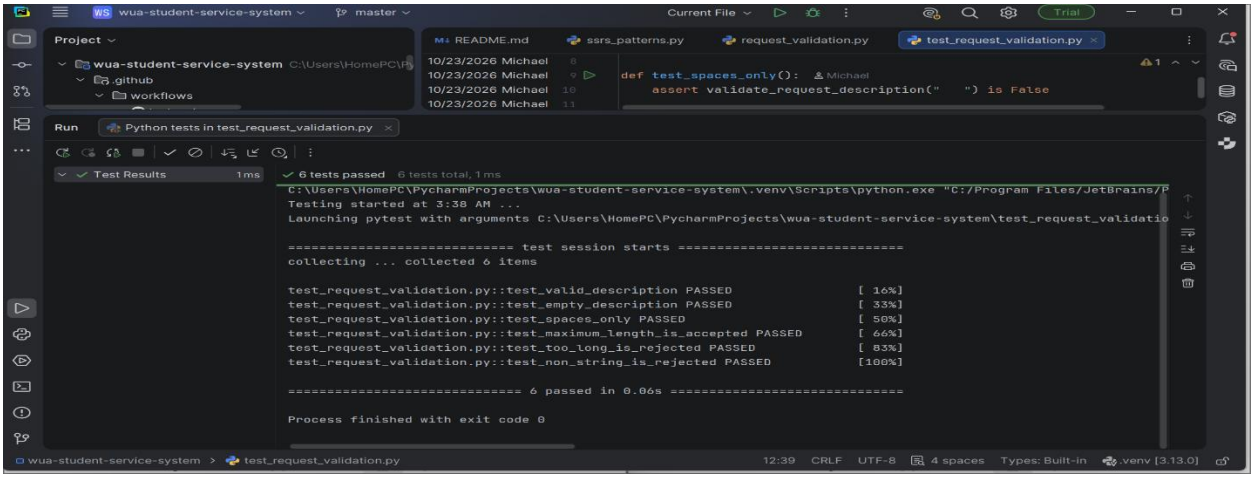
```
python -m pytest -v
```

Expected result: six tests pass.

Failed test



Successful test



Edge-case test plan

Case	Input	Expected result	Reason
Normal	"Broken projector"	True	Non-empty text under limit.
Empty string	""	False	Description is required.
Whitespace only	" "	False	Whitespace is not meaningful content.
Boundary	500 characters	True	Maximum permitted length.
Over boundary	501 characters	False	Prevents overly long input.
Wrong type	None	False	Function handles invalid type safely.

Week 9: CI/CD Pipeline with GitHub Actions

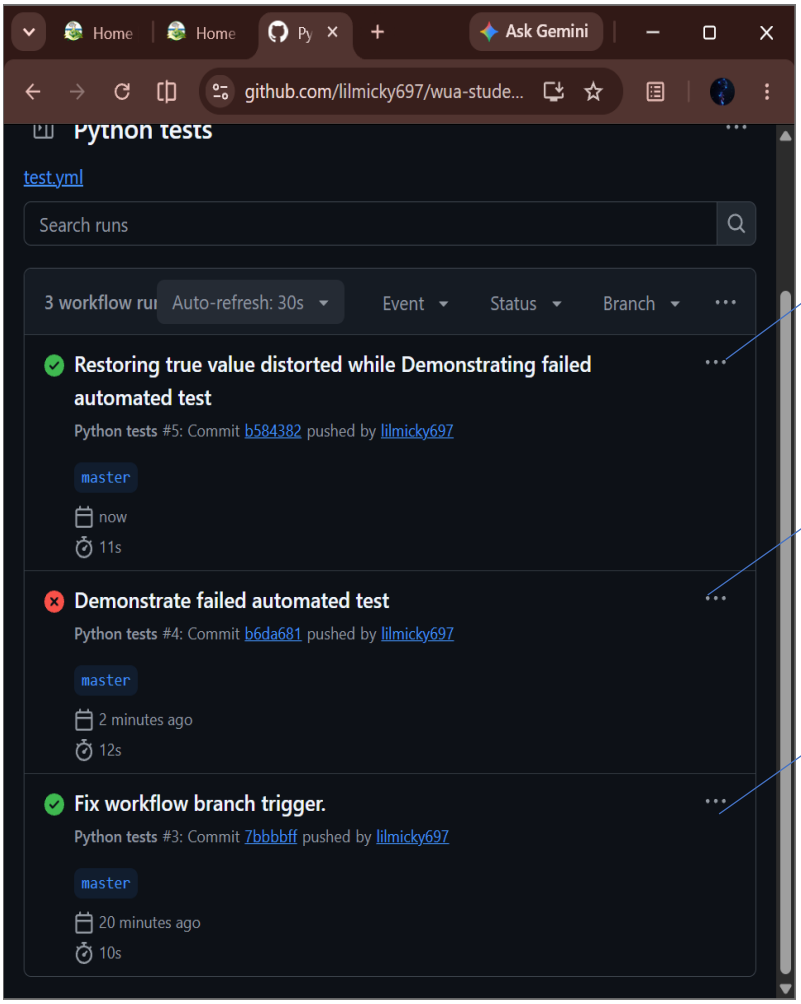
Lab 9: Automating Tests with GitHub Actions

- .github/workflows/test.yml file.

```
name: Python tests

on:
  push:
    branches: [ master ]
  pull_request:
    branches: [ master ]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - name: Check out repository
        uses: actions/checkout@v4
      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - name: Install test dependency
        run: python -m pip install --upgrade pip pytest
      - name: Run unit tests
        run: python -m pytest -v
```



Fix it and verify the pipeline passes.

Break a test intentionally and see the pipeline fail.

Configure it to run pytest or npm test on every push.

Week 10: Project Scheduling & Task Tracking

Lab 10: Sprint Planning with Jira/Trello

Sprint goal: deliver and demonstrate a minimal vertical slice in which a student can submit a valid request, receive a reference number, and see the saved request; automated tests run on GitHub Actions.

Task	Backlog link / story	Estimate	Suggested owner	Definition of done
Create repo, README and Python environment	US01 / project setup	2 points	Developer A	Fresh clone instructions work; code pushed.

Implement request description validation	US02	3 points	Developer A	Validation function and edge-case tests pass.
Create request model/repository prototype	US02, US06	5 points	Developer B	Request data can be saved and retrieved locally.
Build submit-request form and response	US02, US06	5 points	Developer B	Valid submission returns reference; invalid input shows errors.
Implement own-request filtering rule	US03	3 points	Developer C	Tests prove a user cannot retrieve another user's requests.
Add GitHub Actions workflow	CI requirement	2 points	Developer C	Workflow runs and passes on push.
Demonstration and sprint review	All	1 point	Whole team	Show working flow, tests, limitations and next steps.